

Статичный справочник городов для TeamLab MVP

Краткий вывод

Для **TeamLab MVP** я рекомендую не делать жёсткий справочник и не превращать поле `city` в обязательный dropdown. Лучший компромисс для текущего этапа — **текстовое поле + локальный autocomplete по компактному статичному списку**, при этом пользователь всегда может сохранить свой вариант вручную. Такой паттерн соответствует общим UX-практикам autocomplete: компонент должен **подсказывать**, но не обязан ограничивать ввод только словарём. ¹

По размеру справочника для запуска TeamLab оптимальна такая схема:

- **launch core**: около **90–110 городов**;
- **recommended master-list**: около **150 городов**;
- **перебор для MVP** начинается примерно после **500+ городов**.

Граница между “достаточно” и “перебор” здесь проходит не по серверной нагрузке, а по **качеству продукта**: после покрытия крупных российских городов, столиц и самых вероятных русскоязычных relocation/remote-хабов дальнейшее расширение даёт всё меньше пользы, зато резко увеличивает хвост alias-ов, проверок, спорных названий, транслитераций и поддержки. Для России это особенно видно по длинному хвосту городов: Росстат публикует муниципальные оценки численности населения, а вторичные агрегированные списки на базе данных Росстата показывают, что только городов со 100 тыс.+ жителей уже очень много; это плохая точка для “включить всё подряд” в MVP. ²

Практически это означает следующее: **для первого релиза не нужен глобальный или даже “все города СНГ” справочник**. Нужен **сжатый список высокой вероятности выбора**: Россия как приоритет, затем Беларусь и Казахстан, затем столицы и крупнейшие города Армении, Грузии, Кыргызстана, Узбекистана, Азербайджана, Молдовы; отдельно и очень дозированно — Турция, Сербия и Черногория как relocation/remote-страны. Для части этих стран есть прямая статистическая и миграционная логика: например, в Казахстане на начало 2026 года официальная статистика фиксировала почти **2.94 млн русских** и сильную урбанизацию, в Грузии видны крупные миграционные колебания и доступна официальная выгрузка населения по городам, в Армении Госдеп США отмечал релокацию десятков технологических компаний из России, в Турции и Сербии задокументированы крупные русскоязычные relocation-потoki, а в Черногории перепись 2023 года показывает заметную концентрацию населения в нескольких муниципалитетах и наличие российской общины. ³

Методика и продуктовая рамка

На какие страны и регионы имеет смысл смотреть

Если исходить не из абстрактной демографии, а из **русскоязычной TeamLab-аудитории в creative/digital/product**, то покрытие стоит строить слоями.

Первый слой — Россия. Это основной рынок, где autocomplete реально уменьшит число опечаток и визуальный шум в профилях. Здесь приоритет должны получить крупнейшие города, миллионники, региональные центры, университетские и IT-хабы, а также города с высокой узнаваемостью у распределённых команд. Росстат публикует свежие оценки населения по муниципальным образованиям, чего для продуктового отбора более чем достаточно. ⁴

Второй слой — Беларусь и Казахстан. Для Беларуси Белстат публикует официальную статистику по населению в разрезе областей, районов, городов и посёлков городского типа. Для Казахстана официальная статистика по населению и региональным данным доступна у Бюро национальной статистики; отдельно видно, что крупнейшие городские центры — Алматы, Астана и Шымкент — сильно выделяются по масштабу. ⁵

Третий слой — столицы и крупнейшие города стран, где русскоязычные remote-специалисты и команды всё ещё довольно вероятны: Армения, Грузия, Кыргызстан, Узбекистан, Азербайджан, Молдова. Для Грузии у Geostat есть открытые таблицы населения **по городам**, для Узбекистана — открытая таблица постоянного населения по городам, для Азербайджана — официальные демографические таблицы по городам и городским поселениям, для Молдовы — уже опубликованы итоги переписи 2024 года с ранжированием городов. ⁶

Четвёртый слой — только несколько городов Турции, Сербии и Черногории. Их стоит включать не “потому что мир большой”, а потому что есть **продуктовая логика русскоязычной релокации**. У Турции были очень заметные потоки российских резидентов; исследование ZOiS со ссылкой на официальные данные турецкой миграционной службы указывает, что в начале 2024 года у россиян было около **100 тыс. долгосрочных** и **67 тыс. краткосрочных** ВНЖ, что делает Стамбул и Анталию продуктивно оправданными. Reuters по Сербии писал о **30 тыс.+ временно проживающих россиянах** и более **11 тыс. бизнесов**, зарегистрированных в период волны релокации, с явной концентрацией в Белграде. В Черногории официальная перепись показывает, что Подгорица, Никшич и Бар концентрируют почти половину населения страны, а в этнической структуре россияне составили 2.06%; этого достаточно, чтобы держать Подгорицу и ещё 1–2 города максимум во второй волне, но не раздувать список дальше. ⁷

Как я предлагаю отбирать города

Для MVP города стоит отбирать по **пяти простым критериям**, а не по географической полноте.

Во-первых, **население**: в России — в основном крупные города и заметные региональные центры; за пределами России — прежде всего столицы и самые крупные/узнаваемые города. Во-вторых, **административная значимость**: столицы и центры регионов резко повышают вероятность выбора и узнаваемость списка. В-третьих, **релевантность для digital/product/creative-аудитории**: университетские, технологические, сервисные центры лучше работают для TeamLab, чем просто

“крупный промышленный город без цифровой видимости”. Для Грузии, например, это дополнительно поддерживается инфраструктурой Georgian Innovation and Technology Agency, у которой технопарки и центры идут не только по Тбилиси, но и по региональным точкам. ⁸

В-четвёртых, **релокационная значимость**: Тбилиси, Батуми, Ереван, Алматы, Астана, Стамбул, Анталья, Белград, Подгорица — города, где продуктово имеет смысл ожидать русскоязычных remote-пользователей даже при неидеальной демографической строгости. Для Армении это особенно важно: в официальном Investment Climate Statement Госдеп США указывал, что после 2022 года в страну переехали десятки технологических компаний из России, а технологическая рабочая сила заметно выросла. ⁹

В-пятых, **частотность и узнаваемость**. Для autocomplete важна не научная полнота, а вероятность того, что пользователь увидит “свой” вариант без лишних сомнений. Поэтому в MVP полезнее включить, скажем, **Томск, Калининград, Батуми, Анталию и Белград**, чем заполнять список десятками малых райцентров только потому, что они формально города.

Размер справочника и граница между достаточно и перебор

Три размера справочника

size	диапазон	что внутри	плюсы	минусы	кому подходит	рекомендация для Team MVP
minimal	50–100	РФ-миллионники, крупные региональные центры, столицы соседних стран, 2–4 relocation-хаба	очень просто поддерживать; почти нулевая нагрузка; быстро вычистить alias-ы	заметно больше ручного ввода; хуже покрытие Сибири, Севера, Кавказа и соседних стран	очень ранний MVP, когда город — почти декоративное поле	можно, нижняя граница
recommended	150–300	launch core + вторая волна: важные региональные центры РФ, крупные города СНГ, несколько relocation-городов	лучший баланс UX/простоты; хорошо снижает опечатки; покрывает основные сценарии без геобазы	нужно чуть аккуратнее вести alias-ы и порядок сортировки	продукт, который уже хочет “нормально закрыть” профили без тяжёлой инфраструктуры	да, лучший вариант

size	диапазон	что внутри	плюсы	минусы	кому подходит	рекомендация для TeamLab MVP
extended	500-1000	почти все крупные и средние города РФ + расширенный постсоветский контур	высокий recall, меньше ручного ввода	падает качество подсказок, длиннее поддержка, больше неоднозначностей, почти неизбежна потребность в более умном search/ranking	если город скоро станет важным фильтром и источник правится регулярно	нет, не TeamLab

Для TeamLab я бы зафиксировал такую продуктовую позицию: **основной мастер-список — около 150 городов**, но **в первом запуске показывать и поддерживать как “обязательное ядро” примерно 95 городов**. Это даёт чистую MVP-реализацию сейчас и понятную вторую волну без переделки модели данных. Такая схема снижает риск prematurely optimizing taxonomy и хорошо совпадает с тем, что autocomplete должен помогать вводу, а не заменять полноценный геосервис.

1

Примерный размер данных и нагрузка

Если хранить объекты городов в компактном minified JSON с полями уровня `name`, `slug`, `country`, `region`, `aliases`, `priority`, то порядок размеров будет примерно таким:

объём	raw JSON	gzip / brotli	вывод
100 городов	~18–30 KB	~6–10 KB	абсолютно безопасно
250 городов	~45–75 KB	~12–25 KB	безопасно даже для очень маленького проекта
500 городов	~90–150 KB	~25–45 KB	технически всё ещё ок, но уже спорно продуктивно
1000 городов	~180–300 KB	~45–90 KB	сервер выдержит, но для MVP это уже избыточный справочник

Это безопасно для маленького сервера не потому, что “сервер мощный”, а потому что **статический текстовый asset очень хорошо сжимается**; web.dev отдельно рекомендует статическую компрессию для текстовых ресурсов и отмечает, что CDN обычно умеют делать это автоматически.

10

Отсюда практический вывод: **до 150–250 городов справочник спокойно можно хранить на фронте как статичный JSON**. Для вашего контекста — это вообще лучший вариант, потому что он убирает runtime-нагрузку с backend почти полностью. Через backend endpoint список имеет смысл отдавать позднее, если у вас появятся частые правки справочника без фронтенд-релизов, A/B-тесты

подсказок, аналитика по востребованности городов или серверный ranking. Пока этого нет, отдельный endpoint — скорее лишняя движущаяся часть, чем реальная польза.

Рекомендованная стратегия и техническая реализация

MVP-режим

Для TeamLab MVP я рекомендую такую схему.

Поле `city` остаётся **обычным текстовым input**. При вводе **2–3 символов** появляются подсказки из локального списка. Пользователь может выбрать вариант из подсказок, но если нужного города нет, он просто **сохраняет свой текст как есть**. Это соответствует нормальной логике autocomplete: подсказка помогает, но не запрещает свободный ввод. ¹¹

Для локального списка **не нужен debounce**, если поиск идёт по уже загруженному массиву в памяти браузера: там нет сетевого шума и нет смысла искусственно задерживать ответы. Debounce имеет смысл включать позже только для серверного endpoint или внешнего API, где действительно нужно сглаживать частые вызовы; MDN определяет debounce именно как способ объединять слишком частые операции в один вызов. ¹²

Ранжирование подсказок лучше строить так:

1. точное совпадение по `name`;
2. точное совпадение по `alias`;
3. `startsWith` по каноническому имени;
4. `startsWith` по `alias`;
5. `contains` по `alias`;
6. внутри совпавших — сначала `core`, потом `optional`;
7. затем `sort_order` или `product-priority`.

Показывать стоит **не больше 6–8 подсказок**, иначе список начинает визуальнo шуметь и хуже помогает выбору. Это не жёсткая научная цифра, а UX-практика для таких полей: подсказки должны быть быстрыми и управляемыми, а не превращаться в мини-справочник на экране. ¹³

Later-режим

Если later вам понадобится более “умный” autocomplete, переход можно сделать без смены базового контракта данных.

Сейчас вы храните статический список с `slug`, `name`, `aliases`, `country`, `region`, `priority`. Позже можно:

- заменить источник данных на backend endpoint;
- добавить `external_id` и `source`;
- подключить DaData / Yandex Geosuggest / GeoNames только как **дополнительный источник подсказок**, а не как жёсткий контроль поля;

- оставить текущий local JSON как fallback и как словарь частых городов;
- вести аналитику по “manual entries not found”, чтобы понимать, что добавлять во вторую волну.

Если список когда-нибудь переедет в БД, то при объёме **до ~1000 строк** и редких запросах отдельный индекс не обязателен. Но как только появится серверный fuzzy/prefix search, можно добавить простой индекс на нормализованное поле для prefix-поиска, а для похожих строк — trigram-поиск: PostgreSQL прямо документирует `pg_trgm` как модуль для similarity и быстрого поиска похожих строк. ¹⁴

Что делать, если пользовательского города нет в списке

Здесь правило должно быть очень жёстким и очень простым: **не найден — не проблема**.

Нужное поведение:

- не показывать ошибку “город должен быть из справочника”;
- не блокировать сохранение;
- можно показать мягкий сценарий: “Не нашли город? Сохранить как введено”;
- можно логировать такие случаи в аналитику, но не навязывать пользователю правку;
- если пользователь печатает вариант вроде `Moscow`, а в словаре есть `Москва`, предлагать каноническую форму, но **не переписывать автоматически без выбора пользователем**.

Для MVP это важнее любой “чистоты справочника”: вы не делаете геонормативную систему, вы уменьшаете трение при заполнении профиля.

Финальный core-список городов для TeamLab MVP

Ниже — **launch core**: список, который я бы реально запускал в TeamLab в первом релизе. Он покрывает Россию как главный рынок, затем Беларусь и Казахстан, затем наиболее вероятные города для русскоязычной удалённой и relocation-аудитории. Список синтезирован из официальных статистических источников по странам и публичных данных по миграционным/релокационным хамам; для России использована логика “крупный/региональный/университетский/узнаваемый город”, а не попытка включить весь городской хвост. ¹⁵

order	city	country	region	slug	aliases	pr
1	Москва	Россия	Москва	moscow	<code>["Moscow", "мск"]</code>	co
2	Санкт-Петербург	Россия	Санкт-Петербург	saint-petersburg	<code>["Saint Petersburg", "СПб", "Питер"]</code>	co
3	Новосибирск	Россия	Новосибирская область	novosibirsk	<code>["Novosibirsk"]</code>	co

order	city	country	region	slug	aliases	pr
4	Екатеринбург	Россия	Свердловская область	yekaterinburg	["Yekaterinburg", "Екб"]	co
5	Казань	Россия	Татарстан	kazan	["Kazan"]	co
6	Нижний Новгород	Россия	Нижегородская область	nizhny-novgorod	["Nizhny Novgorod", "НН"]	co
7	Самара	Россия	Самарская область	samara	["Samara"]	co
8	Уфа	Россия	Башкортостан	ufa	["Ufa"]	co
9	Челябинск	Россия	Челябинская область	chelyabinsk	["Chelyabinsk"]	co
10	Омск	Россия	Омская область	omsk	["Omsk"]	co
11	Красноярск	Россия	Красноярский край	krasnoyarsk	["Krasnoyarsk"]	co
12	Пермь	Россия	Пермский край	perm	["Perm"]	co
13	Ростов-на-Дону	Россия	Ростовская область	rostov-on-don	["Rostov-on-Don", "Ростов"]	co
14	Краснодар	Россия	Краснодарский край	krasnodar	["Krasnodar"]	co
15	Воронеж	Россия	Воронежская область	voronezh	["Voronezh"]	co
16	Волгоград	Россия	Волгоградская область	volgograd	["Volgograd"]	co

order	city	country	region	slug	aliases	pr
17	Саратов	Россия	Саратовская область	saratov	["Saratov"]	co
18	Тюмень	Россия	Тюменская область	tyumen	["Tyumen"]	co
19	Тольятти	Россия	Самарская область	tolyatti	["Tolyatti"]	co
20	Ижевск	Россия	Удмуртия	izhevsk	["Izhevsk"]	co
21	Барнаул	Россия	Алтайский край	barnaul	["Barnaul"]	co
22	Махачкала	Россия	Дагестан	makhachkala	["Makhachkala"]	co
23	Иркутск	Россия	Иркутская область	irkutsk	["Irkutsk"]	co
24	Хабаровск	Россия	Хабаровский край	khabarovsk	["Khabarovsk"]	co
25	Владивосток	Россия	Приморский край	vladivostok	["Vladivostok"]	co
26	Ярославль	Россия	Ярославская область	yaroslavl	["Yaroslavl"]	co
27	Томск	Россия	Томская область	tomsk	["Tomsk"]	co
28	Калининград	Россия	Калининградская область	kaliningrad	["Kaliningrad"]	co
29	Сочи	Россия	Краснодарский край	sochi	["Sochi"]	co
30	Тула	Россия	Тульская область	tula	["Tula"]	co

order	city	country	region	slug	aliases	pr
31	Рязань	Россия	Рязанская область	ryazan	["Ryazan"]	co
32	Пенза	Россия	Пензенская область	penza	["Penza"]	co
33	Липецк	Россия	Липецкая область	lipetsk	["Lipetsk"]	co
34	Белгород	Россия	Белгородская область	belgorod	["Belgorod"]	co
35	Курск	Россия	Курская область	kursk	["Kursk"]	co
36	Смоленск	Россия	Смоленская область	smolensk	["Smolensk"]	co
37	Брянск	Россия	Брянская область	bryansk	["Bryansk"]	co
38	Тверь	Россия	Тверская область	tver	["Tver"]	co
39	Иваново	Россия	Ивановская область	ivanovo	["Ivanovo"]	co
40	Калуга	Россия	Калужская область	kaluga	["Kaluga"]	co
41	Киров	Россия	Кировская область	kirov	["Kirov"]	co
42	Оренбург	Россия	Оренбургская область	orenburg	["Orenburg"]	co
43	Ульяновск	Россия	Ульяновская область	ulyanovsk	["Ulyanovsk"]	co
44	Набережные Челны	Россия	Татарстан	naberezhnye-chelny	["Naberezhnye Chelny"]	co

order	city	country	region	slug	aliases	pr
45	Астрахань	Россия	Астраханская область	astrakhan	["Astrakhan"]	co
46	Ставрополь	Россия	Ставропольский край	stavropol	["Stavropol"]	co
47	Архангельск	Россия	Архангельская область	arkhangelsk	["Arkhangelsk"]	co
48	Мурманск	Россия	Мурманская область	murmansk	["Murmansk"]	co
49	Петрозаводск	Россия	Карелия	petrozavodsk	["Petrozavodsk"]	co
50	Сыктывкар	Россия	Коми	syktyvkar	["Syktyvkar"]	co
51	Якутск	Россия	Саха	yakutsk	["Yakutsk"]	co
52	Улан-Удэ	Россия	Бурятия	ulan-ude	["Ulan-Ude"]	co
53	Чита	Россия	Забайкальский край	chita	["Chita"]	co
54	Чебоксары	Россия	Чувашия	cheboksary	["Cheboksary"]	co
55	Йошкар-Ола	Россия	Марий Эл	yoshkar-ola	["Yoshkar-Ola"]	co
56	Владимир	Россия	Владимирская область	vladimir	["Vladimir"]	co
57	Вологда	Россия	Вологодская область	vologda	["Vologda"]	co
58	Сургут	Россия	ХМАО	surgut	["Surgut"]	co
59	Кемерово	Россия	Кемеровская область	kemerovo	["Kemerovo"]	co
60	Минск	Беларусь	Минск	minsk	["Minsk"]	co

order	city	country	region	slug	aliases	pr
61	Гомель	Беларусь	Гомельская область	gomel	["Gomel", "Homiel"]	co
62	Брест	Беларусь	Брестская область	breſt	["Brest"]	co
63	Гродно	Беларусь	Гродненская область	grodno	["Grodno", "Hrodna"]	co
64	Витебск	Беларусь	Витебская область	vitebsk	["Vitebsk"]	co
65	Могилёв	Беларусь	Могилёвская область	mogilev	["Mogilev", "Mahilyow"]	co
66	Алматы	Казахстан	Алматы	almaty	["Almaty", "Алма-Ата"]	co
67	Астана	Казахстан	Астана	astana	["Astana", "Нур-Султан"]	co
68	Шымкент	Казахстан	Шымкент	shymkent	["Shymkent"]	co
69	Караганда	Казахстан	Карагандинская область	karaganda	["Karaganda"]	co
70	Актобе	Казахстан	Актюбинская область	aktobe	["Aktobe"]	co
71	Атырау	Казахстан	Атырауская область	atyrau	["Atyrau"]	co
72	Павлодар	Казахстан	Павлодарская область	pavlodar	["Pavlodar"]	co
73	Костанай	Казахстан	Костанайская область	kostanay	["Kostanay"]	co
74	Ереван	Армения	Ереван	yerevan	["Yerevan"]	co

order	city	country	region	slug	aliases	pr
75	Гюмри	Армения	Ширак	gyumri	["Gyumri"]	co
76	Тбилиси	Грузия	Тбилиси	tbilisi	["Tbilisi"]	co
77	Батуми	Грузия	Аджария	batumi	["Batumi"]	co
78	Кутаиси	Грузия	Имеретия	kutaisi	["Kutaisi"]	co
79	Бишкек	Кыргызстан	Бишкек	bishkek	["Bishkek"]	co
80	Ош	Кыргызстан	Ош	osh	["Osh"]	co
81	Ташкент	Узбекистан	Ташкент	tashkent	["Tashkent"]	co
82	Самарканд	Узбекистан	Самаркандская область	samarkand	["Samarkand"]	co
83	Наманган	Узбекистан	Наманганская область	namangan	["Namangan"]	co
84	Фергана	Узбекистан	Ферганская область	fergana	["Fergana"]	co
85	Баку	Азербайджан	Баку	baku	["Baku"]	co
86	Гянджа	Азербайджан	Гянджа-Дашкесан	ganja	["Ganja"]	co
87	Сумгаит	Азербайджан	Абшерон-Хызы	sumqayit	["Sumqayit", "Sumgait"]	co

order	city	country	region	slug	aliases	pr
88	Кишинёв	Молдова	Кишинёв	chisinau	["Chisinau", "Chişinău"]	co
89	Бельцы	Молдова	Бельцы	balti	["Balti", "Bălţi"]	co
90	Стамбул	Турция	Стамбул	istanbul	["Istanbul"]	co
91	Анталья	Турция	Анталья	antalya	["Antalya"]	co
92	Белград	Сербия	Белград	belgrade	["Belgrade", "Beograd"]	co
93	Нови-Сад	Сербия	Южно-Бачский округ	novi-sad	["Novi Sad"]	co
94	Подгорица	Черногория	Подгорица	podgorica	["Podgorica"]	co
95	Измир	Турция	Измир	izmir	["Izmir"]	co

Optional second wave и что не включать

Optional second wave

Ниже — города, которые я бы **не грузил в launch core**, но держал как готовую вторую волну. Этот список доводит мастер-справочник примерно до **150 городов**. Добавлять его имеет смысл не “сразу”, а после первых 1–2 месяцев наблюдения за ручными вводами пользователей. Логика здесь та же: сначала закрываем частотный core, потом добираем реальный спрос. ¹¹

city	country	why optional
Новокузнецк	Россия	большой Кузбасс, но уступает core по digital-частотности

city	country	why optional
Великий Новгород	Россия	областной центр, полезен для северо-западного покрытия
Псков	Россия	областной центр, но не обязателен на старте
Кострома	Россия	региональный центр второй волны
Орёл	Россия	покрытие ЦФО после запуска
Тамбов	Россия	частотность ниже, чем у core-центров
Курган	Россия	полезен для геопокрытия, но не обязателен
Саранск	Россия	региональный центр второй волны
Стерлитамак	Россия	крупный город Башкортостана вне столицы
Магнитогорск	Россия	большой индустриальный город, но ниже приоритетом
Владикавказ	Россия	значимая столица региона, но не launch-critical
Нальчик	Россия	столица региона, полезна позже
Грозный	Россия	столица региона, но для MVP можно отложить
Майкоп	Россия	небольшой столичный центр, не обязателен в первой волне
Благовещенск	Россия	дальневосточное покрытие второй волны
Южно-Сахалинск	Россия	полезный удалённый хаб, но низкая частота
Петропавловск-Камчатский	Россия	экстремальная география, низкая частота
Ноябрьск	Россия	северный рабочий город, имеет смысл позже
Нижневартовск	Россия	заметный северный узел второй волны
Ханты-Мансийск	Россия	административно значим, но небольшой
Абакан	Россия	столица региона, но не launch-critical
Иннополис	Россия	очень релевантен по IT, но маленький и нишевый
Новороссийск	Россия	relocation/remote потенциал юга
Пятигорск	Россия	узнаваемый кавказский city-node
Горно-Алтайск	Россия	столица региона, но низкая частота
Черкесск	Россия	столица региона, но низкая частота
Северодвинск	Россия	полезен только при наблюдаемом спросе

city	country	why optional
Комсомольск-на-Амуре	Россия	дальневосточное покрытие второй волны
Бийск	Россия	заметный город Алтайского края, но ниже частоты
Череповец	Россия	крупный не-столичный центр
Обнинск	Россия	техно- и научный город; хорош, если видите соответствующую аудиторию
Дубна	Россия	сильный научный бренд; хороша как нишевый optional
Барановичи	Беларусь	заметный не-столичный город, но не обязателен в core
Бобруйск	Беларусь	частотный городской second wave
Актау	Казахстан	каспийский релокационный и деловой хаб
Усть-Каменогорск	Казахстан	крупный восточный центр
Семей	Казахстан	заметный городской центр второй волны
Уральск	Казахстан	важный западный центр
Ванадзор	Армения	третий город страны, полезен позже
Рустави	Грузия	крупный город рядом с Тбилиси
Зугдиди	Грузия	региональный центр; у GITA есть технопарк
Джалал-Абад	Кыргызстан	вторая волна после Бишкека и Оша
Каракол	Кыргызстан	заметный город, но не обязателен для запуска
Бухара	Узбекистан	крупный и узнаваемый город
Андижан	Узбекистан	важный центр Ферганской долины
Нукус	Узбекистан	столица Каракалпакстана, полезен позже
Нахичевань	Азербайджан	отдельный значимый центр, но не core
Ленкорань	Азербайджан	региональная значимость второй волны
Кагул	Молдова	логичный третий город Молдовы
Анкара	Турция	важная столица, но для русскоязычного MVP менее частотна, чем Стамбул/Анталья
Аланья	Турция	русскоязычный relocation-сценарий, но нишевое Антальи
Ниш	Сербия	логичный третий город Сербии
Будва	Черногория	сильный expat/remote-кейс, но не обязателен на старте

city	country	why optional
Тиват	Черногория	expat-pattern, но для launch уже лишний

Что не стоит включать в MVP

В MVP я бы **не включал** следующие группы.

Не стоит включать **все города России 100k+ просто по населению**. Это быстро превращает autocomplete в тяжёлый “полу-справочник” без заметного UX-выигрыша для TeamLab. Росстат и вторичные списки на его базе показывают, что такой хвост большой, а для вашего продукта он сейчас не нужен. ²

Не стоит включать **города мира без специальной продуктовой логики**: Дубай, Лимасол, Тель-Авив, Берлин, Варшава, Прага, Лиссабон, Барселона, Тбилиси уже есть, но не нужно идти дальше в Европу и MENA только потому, что там “могут быть релоканты”. Это делает словарь бесформенным и размывает фокус русского интерфейса.

Не стоит включать **все спутники Москвы и Петербурга**. Балашиха, Химки, Мытищи, Одинцово, Красногорск и т.п. действительно частотны в общей популяции, но для MVP проще дать человеку сохранить их вручную, чем начинать отдельный класс “агломерационных городов”, который неизбежно раздует справочник.

Я бы также **не включал геополитически спорные и чувствительные территории как часть launch taxonomy**, если вы не готовы отдельно проработать их юридическую и продуктовую трактовку. Для MVP это лишняя сложность.

Нормализация и UX-логика

Правила нормализации

Для autocomplete нужна **нормализация поиска**, но не нужно навязывать её как жёсткое значение, записываемое поверх ввода пользователя.

Рекомендованный набор правил такой:

- приводить строку к lower-case;
- обрезать внешние пробелы;
- схлопывать повторные пробелы;
- заменять **ё** на **е**;
- унифицировать дефисы, тире и пробелы вокруг них;
- удалять “шумовые” точки в кратких формах вроде **спб.** / **мск.** при поиске;
- поддерживать кириллицу и латиницу через **aliases**;
- хранить разговорные и исторические варианты там, где это реально встречается.

Минимальный набор нормализации по примерам:

- Москва → aliases: Moscow, мск
- Санкт-Петербург → aliases: Saint Petersburg, St Petersburg, СПб, Питер
- Нижний Новгород → aliases: Nizhny Novgorod, НН
- Алматы → aliases: Almaty, Алма-Ата
- Астана → aliases: Astana, Нур-Султан
- Тбилиси → aliases: Tbilisi
- Ереван → aliases: Yerevan

Важно: `aliases` должны использоваться **для матчинга**, а не обязательно для отображения. Отображаемое каноническое имя — это `name`. Если пользователь выбрал подсказку Санкт-Петербург, в поле лучше сохранить каноническое русскоязычное имя. Если он сам ввёл Saint Petersburg и не выбрал подсказку, можно сохранить именно введенный вариант и не “исправлять” его силой.

Рекомендованная UX-логика

Поведение поля я бы зафиксировал так:

- поле остаётся текстовым;
- подсказки появляются с **2 символов** для кириллицы и латиницы; с 3 символов — если хотите ещё сильнее убрать шум;
- пользователь может выбрать город из подсказок мышью или клавиатурой;
- если города нет, пользователь сохраняет свой вариант;
- валидация по словарю **не жёсткая**;
- на submit нельзя блокировать малые города и редкие варианты;
- при выборе из списка можно подставлять каноническое значение;
- при ручном вводе лучше сохранить текст пользователя как есть.

Это хорошо согласуется с тем, как рекомендуются autocomplete-компоненты: они поддерживают свободный ввод и работают как soft guidance, а не как запрет на произвольное значение. ¹¹

JSON-структура и пример файла

Рекомендуемая структура данных

Для TeamLab я бы не усложнял модель сверх необходимого. Базовая структура, которую вы предложили, уже хорошая. Я бы добавил только то, что реально поможет в ранжировании и эволюции:

```
{
  "name": "Москва",
  "slug": "moscow",
  "country": "Россия",
  "country_code": "RU",
```

```
"region": "Москва",
"aliases": ["Moscow", "мск"],
"priority": "core",
"sort_order": 10,
"population_group": "1m_plus",
"is_capital": true
}
```

Что здесь полезно:

- `slug` — будущий стабильный ключ для перехода на endpoint или внешний источник;
- `country_code` — пригодится позже для фильтров и аналитики;
- `sort_order` — чтобы не держать ranking в коде;
- `population_group` — грубый продуктовый сигнал для сортировки;
- `is_capital` — маленький полезный флаг для ranking/helper-логики.

Что пока **не нужно**:

- координаты;
- отдельные сущности страны/региона с полноценной геомodelью;
- timezone для всех городов;
- сложные типы населённых пунктов;
- backend-lookup-таблицы под все варианты написания.

Где хранить список

Для MVP лучшее решение — **статичный JSON на фронте**. Либо в бандле, либо отдельным asset, который подгружается лениво при фокусе или первом открытии профиля. Это убирает почти всю нагрузку с сервера и соответствует вашему ограничению “на сервере уже крутятся ещё два проекта”. Текстовые статические ресурсы хорошо сжимаются и обычно вообще не создают существенной инфраструктурной проблемы. ¹⁶

Через backend endpoint имеет смысл отдавать список только если:

- вы хотите обновлять справочник без деплоя фронта;
- вам нужна аналитика по запросам autocomplete на сервере;
- список станет заметно больше;
- появятся разные словари по локали/стране;
- вы захотите объединить локальные города и внешний API в один ranking.

Нужен ли backend index

Если список живёт на фронте — **индекс на backend не нужен вообще**.

Если список временно лежит в БД, но его там до 1000 записей и autocomplete не серверный, отдельный индекс тоже, скорее всего, преждевременен. Но если позже вы делаете серверный prefix/fuzzy search, то:

- для prefix-поиска по нормализованному имени достаточно обычного индекса подходящего типа;
- для fuzzy/similarity поиска в PostgreSQL уже логично смотреть в сторону `pg_trgm`, который официально предназначен для similarity и быстрого поиска похожих строк. ¹⁷

Итоговый JSON-пример

Нижеследующий — пример того, как может выглядеть файл на 15 городов. Он уже совместим с подходом “сейчас локально, позже — endpoint/API”.

```
[
  {
    "name": "Москва",
    "slug": "moscow",
    "country": "Россия",
    "country_code": "RU",
    "region": "Москва",
    "aliases": ["Moscow", "мск"],
    "priority": "core",
    "sort_order": 10,
    "population_group": "1m_plus",
    "is_capital": true
  },
  {
    "name": "Санкт-Петербург",
    "slug": "saint-petersburg",
    "country": "Россия",
    "country_code": "RU",
    "region": "Санкт-Петербург",
    "aliases": ["Saint Petersburg", "St Petersburg", "СПб", "Питер"],
    "priority": "core",
    "sort_order": 20,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Новосибирск",
    "slug": "novosibirsk",
    "country": "Россия",
    "country_code": "RU",
    "region": "Новосибирская область",
    "aliases": ["Novosibirsk"],
    "priority": "core",
```

```

    "sort_order": 30,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Екатеринбург",
    "slug": "yekaterinburg",
    "country": "Россия",
    "country_code": "RU",
    "region": "Свердловская область",
    "aliases": ["Yekaterinburg", "Екб"],
    "priority": "core",
    "sort_order": 40,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Казань",
    "slug": "kazan",
    "country": "Россия",
    "country_code": "RU",
    "region": "Татарстан",
    "aliases": ["Kazan"],
    "priority": "core",
    "sort_order": 50,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Нижний Новгород",
    "slug": "nizhny-novgorod",
    "country": "Россия",
    "country_code": "RU",
    "region": "Нижегородская область",
    "aliases": ["Nizhny Novgorod", "НН"],
    "priority": "core",
    "sort_order": 60,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Томск",
    "slug": "tomsk",
    "country": "Россия",
    "country_code": "RU",
    "region": "Томская область",
    "aliases": ["Tomsk"],
    "priority": "core",

```

```

    "sort_order": 70,
    "population_group": "250k_1m",
    "is_capital": false
  },
  {
    "name": "Калининград",
    "slug": "kaliningrad",
    "country": "Россия",
    "country_code": "RU",
    "region": "Калининградская область",
    "aliases": ["Kaliningrad"],
    "priority": "core",
    "sort_order": 80,
    "population_group": "250k_1m",
    "is_capital": false
  },
  {
    "name": "Минск",
    "slug": "minsk",
    "country": "Беларусь",
    "country_code": "BY",
    "region": "Минск",
    "aliases": ["Minsk"],
    "priority": "core",
    "sort_order": 110,
    "population_group": "1m_plus",
    "is_capital": true
  },
  {
    "name": "Алматы",
    "slug": "almaty",
    "country": "Казахстан",
    "country_code": "KZ",
    "region": "Алматы",
    "aliases": ["Almaty", "Алма-Ата"],
    "priority": "core",
    "sort_order": 120,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Астана",
    "slug": "astana",
    "country": "Казахстан",
    "country_code": "KZ",
    "region": "Астана",
    "aliases": ["Astana", "Нур-Султан"],
    "priority": "core",

```

```

    "sort_order": 130,
    "population_group": "1m_plus",
    "is_capital": true
  },
  {
    "name": "Ереван",
    "slug": "yerevan",
    "country": "Армения",
    "country_code": "AM",
    "region": "Ереван",
    "aliases": ["Yerevan"],
    "priority": "core",
    "sort_order": 140,
    "population_group": "1m_plus",
    "is_capital": true
  },
  {
    "name": "Тбилиси",
    "slug": "tbilisi",
    "country": "Грузия",
    "country_code": "GE",
    "region": "Тбилиси",
    "aliases": ["Tbilisi"],
    "priority": "core",
    "sort_order": 150,
    "population_group": "250k_1m",
    "is_capital": true
  },
  {
    "name": "Стамбул",
    "slug": "istanbul",
    "country": "Турция",
    "country_code": "TR",
    "region": "Стамбул",
    "aliases": ["Istanbul"],
    "priority": "core",
    "sort_order": 160,
    "population_group": "1m_plus",
    "is_capital": false
  },
  {
    "name": "Белград",
    "slug": "belgrade",
    "country": "Сербия",
    "country_code": "RS",
    "region": "Белград",
    "aliases": ["Belgrade", "Beograd"],
    "priority": "core",

```

```
"sort_order": 170,  
"population_group": "1m_plus",  
"is_capital": true  
}  
]
```

Итоговая рекомендация в одном предложении: **для TeamLab MVP запускайте текстовое поле с локальным autocomplete по ядру примерно в 95 городов, держите мастер-список около 150 городов, не делайте жёсткую валидацию, храните данные как статичный JSON с `slug` и `aliases`, а вторую волну добавляйте только по реальным manual-entry данным пользователей.** Это даст максимум UX-пользы при минимуме таксономической и инфраструктурной сложности. ¹⁸

1 11 18 Autocomplete - CMS Design System

<https://design.cms.gov/components/autocomplete/>

2 4 15 Демография

https://rosstat.gov.ru/folder/12781?utm_source=chatgpt.com

3 Population of the Republic of Kazakhstan by gender and ...

https://stat.gov.kz/en/industries/social-statistics/demography/publications/478310?utm_source=chatgpt.com

5 Население

https://www.belstat.gov.by/ofitsialnaya-statistika/ssrd-mvf_2/metadannye/realnyi-sektor/naselenie_3/

6 Population - National Statistics Office of Georgia

<https://www.geostat.ge/en/modules/categories/41/population>

7 Turkey: A New Hub for Migration from Russia

<https://www.zois-berlin.de/en/publications/zois-spotlight/turkey-a-new-hub-for-migration-from-russia>

8 TechnoPark and Innovation Centres

<https://gita.gov.ge/en/regions>

9 2024 Investment Climate Statements - Armenia

https://www.state.gov/reports/2024-investment-climate-statements/armenia?utm_source=chatgpt.com

10 16 Optimize the encoding and transfer size of text-based assets | Articles | web.dev

<https://web.dev/articles/optimizing-content-efficiency-optimize-encoding-and-transfer>

12 Debounce - Glossary - MDN Web Docs

https://developer.mozilla.org/en-US/docs/Glossary/Debounce?utm_source=chatgpt.com

13 Site Search Suggestions

https://www.nngroup.com/articles/site-search-suggestions/?utm_source=chatgpt.com

14 PostgreSQL: Documentation: 18: F.35. pg_trgm — support for similarity of text using trigram matching

<https://www.postgresql.org/docs/current/pgtrgm.html>

17 Documentation: 18: 11.2. Index Types

https://www.postgresql.org/docs/current/indexes-types.html?utm_source=chatgpt.com